

المحاضرة 5

بنى التحكم: الجزء الأول

د. سهيل الحمود

د. أسامه ناصر

2025-2026

ماهي بنى التحكم

- عرفنا البرنامج على أنه مجموعة من التعليمات يتم تنفيذها بشكل متتال.
- إنما هنالك مجموعة من التفاصيل الإضافية
- بفرض لدينا جزء من الكود يتم تنفيذه إذا تحقق شرط معين
- أو لدينا جزء من الكود يتم تنفيذه عدد من المرات
- ما الحل في هذه الحالة؟
- الحل يكمن في بنى التحكم
- هي كتل برمجية تسمح لنا بالتحكم بتسلسل تنفيذ التعليمات
- تقسم لشرطية وتكرارية

ماهي بنى التحكم

- البنى الشرطية تمتلك الأنواع التالية:
 - if
 - if else
 - ()?:
 - switch case

البنى الشرطية

if

- في هذه البنية عند تحقق الشرط ننفذ محتوى الشرط، ونتابع بعدها التنفيذ

1. يتم في البداية دراسة قيمة x ضمن عبارة if

2. في حال تحقق الشرط، يتم تنفيذ الكود ضمن كتلة if

3. يتم تنفيذ الكود التالي للكتلة بشكل طبيعي

```
#include<iostream>
using namespace std;
int main(){
    int x;
    cin>>x;//x=20
    if(x>10){
        cout<<"Bigger than 10" <<endl;
    }
    cout<<"The value is " <<x<<endl;
    return 0;}
```

البنى الشرطية

if

- في هذه البنية عند تحقق الشرط ننفذ محتوى الشرط، ونتابع بعدها التنفيذ

1. يتم في البداية دراسة قيمة x ضمن عبارة `if`

2. في حال لم يحقق الشرط، يتم تجاهل الكود ضمن كتلة `if`

3. يتم تنفيذ الكود التالي للكتلة بشكل طبيعي

```
#include<iostream>
using namespace std;
int main(){
    int x;
    cin>>x;//x=2
    if(x>10){
        cout<<"Bigger than 10" <<endl;
    }
    cout<<"The value is " <<x<<endl;
    return 0;}
```

البنى الشرطية

if

عمليات المقارنة

- توجد لدينا عمليات المقارنة التالية في ++C
 - '=' تعني مقارنة المساواة (هل الطرفان متساويان أم لا)
 $x=3$
 - '!=' تعني مقارنة عدم المساواة (هل الطرفان غير متساويان أم لا)
 $x\neq 3$
 - '<' تعني مقارنة أصغر (هل الطرف الأيسر أصغر من الأيمن)
 $x<3$
 - '<=' تعني مقارنة أصغر أو يساوي (هل الطرف الأيسر أصغر أو يساوي الأيمن)
 $x\leq 3$

البنى الشرطية

if

عمليات المقارنة

- توجد لدينا عمليات المقارنة التالية في ++C
 - '>' تعني مقارنة أكبر (هل الطرف الأيسر أكبر من الأيمن)
 $x > 3$
 - '> =' تعني مقارنة أكبر أو يساوي (هل الطرف الأيسر أكبر أو يساوي الطرف الأيمن)
 $x \geq 3$

البنى الشرطية

if

عمليات المقارنة

• أمثلة

```
#include<iostream>
using namespace std;
int main(){
    int x; cin>>x;
    if(x<10) cout<<"<10";
    if(x≤10) cout<<"≤10";
    if(x>10) cout<<">10";
    if(x≥10) cout<<"≥10";
    if(x=10) cout<<"=10";
    if(≠>10) cout<<"≠10";
    return 0;}
```


البنى الشرطية

if

- نريد حساب مربع رقم إذا فقط إذا كان أكبر أو يساوي 2

```
#include<iostream>
using namespace std;
int main(){
    int x;
    if(x ≥ 2) cout<<x*x<<endl;
    return 0;
}
```

البنى الشرطية

if

- نريد حساب الجذر التربيعي لعدد المستخدم

```
#include<iostream>
#include<cmath>
using namespace std;
int main(){
    int x;
    cin>>x;
    if(x>0) cout<<sqrt(x)<<endl;
    return 0;
}
```

البنى الشرطية

if

- نريد تحويل حرف كبير لصغير إذا كان الحرف كبير (يدخل المستخدم)

```
#include<iostream>
using namespace std;
int main(){
    char c;
    cin>>c;
    if(c<'a') c--='a'-'A';
    cout<<c<<endl;
    return 0;
}
```

البنى الشرطية

if

- هل توجد طريقة لتنفيذ أكثر من تعليمة ضمن جسم تعليمة if؟

```
#include<iostream>
using namespace std;
int main(){
    int x;
    cin>>x;
    if(x<10){
        cout<<"the value"<<endl;
        cout<<x;
    }
    return 0;
}
```

البنى الشرطية

if

```
#include<iostream>
using namespace std;
int main(){
    int x;
    cin>>x;
    if(x<10){
        int y=x*10;
        cout<<y<<endl;
    }
    cout<<x+y<<endl;//is this correct?
    return 0
}
```

- هل يا ترى السطر 10 صحيح أم لا؟
- يظهر لنا خطأ في المترجم (دلالي لأن المتحول غير موجود)، السبب أن المتحول معرف ضمن كتلة if وليس ضمن كتلة main
- أي أنه ضمن مجال رؤية فرعي من المجال الرئيسي، وبالتالي لا يراه الرئيسي

البنى الشرطية

if else

- في هذه البنية يتم تنفيذ كتلة if في حال تحقق الشرط أو كتلة else في حال لم يتحقق الشرط

```
#include<iostream>
using namespace std;
int main(){
    int x=0; cin>>x;
    if(x>10) cout<<"bigger than 10"<<endl;
    else cout<<"smaller than or equal to 10"<<endl;
    return 0;
}
```

البنى الشرطية

if else

- لنفترض أن قيمة $x=4$ هذا يعني أن كتلة if ستنفذ
- لنفترض أن قيمة $x=400$ هذا يعني أن كتلة else ستنفذ

```
#include<iostream>
using namespace std;
int main(){
    int x=0; cin>>x;
    if(x>10) cout<<"bigger than 10"<<endl;
    else cout<<"smaller than or equal to 10"
    <<endl;
    return 0;
}
```

البنى الشرطية

if else

- نريد كتابة برنامج يعمل على حل معادلة درجة 2 بالاعتماد على مفهوم المميز

```
#include<iostream>
#include<cmath>
using namespace std;
int main(){
    double a,b,c;
    cin>>a>>b>>c;
    double delta=b*b-4*a*c;
    if(delta >0){
        cout<<"x1="<<(-b-sqrt(delta))/(2*a)
<<endl;
        cout<<"x2="<<(-b+sqrt(delta))/(2*a)
<<endl;
    }
```

```
else if(delta==0)
    cout<<"x1=x2="<< -b/(2*a)<<endl;
else
    cout<<"complex root"<<endl;
return 0;
}
```


البنى الشرطية

if else

- في حال كانت قيمة `delta` أكبر من 0 للمعادلة جذرين
- وإلا في حال كنت قيمة `delta` تساوي 0 للمعادلة جذر مضاعف
- وإلا حل المعادلة في مجموعة الأعداد العقدية

البنى الشرطية

else if

- هل يمكن استخدام if بعد else ؟
- نعم يمكن ذلك، ما يلي تعليمة else هو تعليمات ++c وبالتالي يمكن أن تكون تعليمة if جديدة

الشروط المركبة

- هل يمكن أن يتضمن شرط if أكثر من شرط
- مثلاً نريد حساب مساحة مستطيل
- يقوم المستخدم بإدخال الأبعاد w,h
- في حال كنت قيمة البعدين موجبة نحسب المساحة
- الحل الأول

```
#include<iostream>
using namespace std;
int main(){
    int w,h; cin>>w>>h;
    if(w>0)
        if(h>0)
            cout<<h*w<<endl;
    return 0;
}
```

الشروط المركبة

- هل يمكن أن يتضمن شرط if أكثر من شرط
- مثلاً نريد حساب مساحة مستطيل
- يقوم المستخدم بإدخال الأبعاد w,h
- في حال كنت قيمة البعدين موجبة نحسب المساحة
- الحل الثاني

```
#include<iostream>
using namespace std;
int main(){
    int w,h; cin>>w>>h;
    if(w>0 && h>0)
        cout<<h*w<<endl;
    return 0;
}
```

الشروط المركبة

معاملات تركيب الشروط

&&		&	B	A
0	0	0	0	0
0	0	0	1	0
0	0	0	0	1
1	1	1	1	1

الشروط المركبة

معاملات تركيب الشروط

- مالفرق بين $\&$ و $\&\&$
- الفرق مرتبط بالأداء
- $\&$
- يتم تقييم الطرفين ثم حساب عملية التقاطع (الضرب المنطقي)
- $A \cap B$
- $\&\&$
- في حال كان الطرف A يساوي 0 هذا يعني أنه بغض النظر عن قيمة B فإن القيمة ستكون 0 وبالتالي لاداعي لإجراء الضرب المنطقي
- في حال كان الطرف A يساوي 1 هذا يعني احتمال أن يكون B يساوي 1 وبالتالي ناتج الضرب المنطقي 1 وعليه يتم تنفيذ الضرب في هذه الحالة

الشروط المركبة

معاملات تركيب الشروط

- مالفرق بين $|$ و $||$
- الفرق مرتبط بالأداء
- $|$
- يتم تقييم الطرفين ثم حساب عملية الاجتماع (الجمع المنطقي)
- $A \cup B$
- $||$
- في حال كان الطرف A يساوي 1 هذا يعني أنه بغض النظر عن قيمة B فإن القيمة ستكون 1 وبالتالي لاداعي لإجراء الجمع المنطقي
- في حال كان الطرف A يساوي 0 هذا يعني احتمال أن يكون B يساوي 1 وبالتالي ناتج الجمع المنطقي 1 وعليه يتم تنفيذ الجمع في هذه الحالة

الشرط السريع

- هي عملية المقارنة الشرطية القائمة على ؟
- مثال

```
#include<iostream>
using namespace std;
int main(){
    int n;
    cin>>n;
    cout<<((n%2==0)?"Even":"Odd")
        <<endl;
    return 0;
}
```

```
#include<iostream>
using namespace std;
int main(){
    int n;
    cin>>n;
    if(n%2==0)
        cout<<"Even"<<endl;
    else
        cout<<"Odd"<<endl;
    return 0;
}
```


البنى الشرطية

بنية Switch Case

- هي بنية خاصة تسمح لنا باستبدال مجموعة من عبارات if else

```
#include<iostream>
using namespace std;
int main(){
    int n;cin>>n;
    switch(n){
        case 0:
            cout<<"the value is 0"<<endl;
            break;
        case 1:
            cout<<"the value is 1"<<endl;
        default:
            cout<<"unknown value"<<endl; }    return 0;}
```

البنى الشرطية

بنية Switch Case

```
#include<iostream>
using namespace std;
int main(){
int n;cin>>n;
switch(n){
    case 0:
        cout<<"0"<<endl;
        break;
    case 1:
        cout<<"1"<<endl;
    default:
        cout<<"unknown"<<endl;}return 0;}
```

- تعريف على أي متحول ستعمل عبارة switch
- في حال كانت القيمة 0
- منع متابعة التنفيذ وإنهاء كتلة switch
- في حال كانت القيمة 1
- عدم وجود break يعني تنفيذ الكتلة التالية ضمن switch بعد انتهاء الكتلة الحالية
- الكتلة التي يتم تنفيذها في حال عدم تحقق شرط أي من الكتل السابقة

ملاحظة هامة

نسخة تفاعلية من المحاضرات متوفرة على الرابط : <https://ossnass.github.io/Programmin-1> (مع مراعاة حالة الأحرف)

